

Nonexistence in the $N = 4$, $R = 2$ Knights-and-Rooks Problem

Self-contained proof and independently checkable certificate

13 September 2026

Abstract

There is no finite placement in which every knight attacks exactly four rooks and no knights, while every rook sees exactly two knights and no rooks. A rightmost-knight argument proves a stronger result: if every rook sees exactly two knights and no rooks, then some knight attacks at most three rooks, even when knight–knight attacks are permitted. A separately encoded necessary local relaxation is also proved unsatisfiable by a small, independently checked certificate. The universal conclusion does not rely on exhausting boards of bounded size.

1 Exact model and result

Let $\mathcal{K}, \mathcal{R} \subset \mathbb{Z}^2$ be finite, nonempty, disjoint sets. Their elements are knight and rook squares, respectively. A knight at (x, y) attacks occupied squares whose absolute coordinate differences from (x, y) are 1 and 2. It jumps over intervening pieces.

For a rook, inspect each of the four open orthogonal rays and retain only its first occupied square, when one exists. The rook condition is that exactly two such first squares contain knights and none contain rooks. Consequently, exactly two rays are occupied and the other two rays are *entirely empty*. Attacks are directed; no balance between the two kinds of outgoing attack counts is assumed.

Theorem 1. *If every rook sees exactly two knights and no rooks, then some knight attacks at most three rooks. This holds without a prohibition on knight–knight attacks.*

The requested placement would give every knight four rook targets, so Theorem 1 rules it out.

2 Universal geometric proof

Assume for contradiction that every knight attacks at least four rooks.

A rightmost knight bounds all pieces. Every rook has x -coordinate at most the maximum knight x -coordinate. Indeed, a rook strictly to the right of all knights could see a knight only to its west, giving at most one knight target. Translate a rightmost knight to

$$A = (0, 0).$$

Every square with $x > 0$ is therefore empty. The finiteness assumption is used here to choose a rightmost knight, not to impose a board-size bound.

Four forced rooks and their blockers. Only four knight-move destinations of A have $x \leq 0$. All four must be rooks:

$$U = (-2, 1), \quad V = (-1, 2), \quad W = (-1, -2), \quad Z = (-2, -1).$$

The rooks U, Z must have a knight between them; their sole intervening square is

$$C = (-2, 0),$$

which is consequently a knight. Similarly, V, W require a knight at at least one of

$$B_+ = (-1, 1), \quad B_0 = (-1, 0), \quad B_- = (-1, -1).$$

The middle possibility is impossible. If B_0 were a knight, its two knight-move destinations with $x = 1$ would be empty. Each of its other six destinations is immediately horizontally adjacent to a forced rook:

destination from B_0	adjacent forced rook
$(-3, 1), (-3, -1)$	U, Z
$(-2, 2), (0, 2)$	V
$(-2, -2), (0, -2)$	W

None of these six destinations can be a rook: two horizontally adjacent rooks would see each other. Thus a knight at B_0 would attack no rooks, a contradiction. At least one of B_+, B_- is a knight. Reflect in $y = 0$, if needed, so that B_+ is a knight. This reflection resolves a proved two-case alternative; it is not an assumed symmetry of the placement.

The upper blocker forces a right-edge rook. The rook U already sees C immediately south and B_+ immediately east. Its north and west rays must be entirely empty. In particular, $(-2, 2)$ and $(-2, 3)$ are empty. Also $(-3, 2)$ cannot be a rook, because it would see $V = (-1, 2)$ across empty $(-2, 2)$.

Of the eight knight-move destinations of B_+ , only

$$(-3, 0), \quad (-2, -1), \quad (0, -1), \quad (0, 3)$$

can therefore contain rooks. Two other destinations have $x > 0$, and the remaining two were excluded above. All four listed squares must be rooks. In particular,

$$X = (0, -1)$$

is a rook. The rooks Z, X have only one intervening square, so $B_- = (-1, -1)$ is now forced to be a knight.

The lower blocker has at most three targets. The rook Z sees C immediately north and B_- immediately east, so its south ray is empty. In particular, $(-2, -2)$ and $(-2, -3)$ are empty. Therefore $(-3, -2)$ cannot be a rook, because it would see $W = (-1, -2)$ across empty $(-2, -2)$.

The rook X sees A immediately north and B_- immediately west. Its south ray is empty, so $(0, -3)$ is empty as well. The full list of knight-move destinations of B_- is

$$(-3, -2), (-3, 0), (-2, -3), (-2, 1), \\ (0, -3), (0, 1), (1, -2), (1, 0).$$

The last two are in the empty half-plane $x > 0$, and $(-3, -2)$, $(-2, -3)$, $(0, -3)$ were just excluded. At most three rook targets remain:

$$(-3, 0), \quad (-2, 1), \quad (0, 1).$$

This contradicts the lower bound of four for the knight B_- , proving the theorem. $\square$

No step uses the absence of knight–knight attacks, a fixed piece inventory, board colors, legal-game reachability, or reciprocity between attacks.

3 A separately checkable finite obstruction

The proof above is sufficient on its own. The following computation was used to try to falsify its local reasoning and to provide a checkable certificate.

3.1 Why this is not bounded-board exhaustion

After the proved rightmost-knight reduction, retain only the window

$$D = \{-3, -2, -1, 0\} \times \{-3, -2, -1, 0, 1, 2, 3\}$$

and the four distinguished positions

$$T = \{(0, 0), (-1, 0), (-1, 1), (-1, -1)\}.$$

Every possible rook target of a knight at a position in T lies either in D or in the already-proved empty half-plane $x > 0$. Consequently, their knight-degree requirements can be checked locally without assuming anything about other knights. Pieces outside D are otherwise permitted.

A local rook has at most two locally occupied rays: outside pieces cannot remove a ray that already contains a local occupied square. Every aligned pair of rooks in D must have a knight between them; all intervening squares lie in D . Thus any hypothetical finite counterexample induces an assignment satisfying the following *necessary relaxation*.

3.2 Complete encoding

For each $p \in D$, let K_p, R_p, O_p be Boolean variables for knight, rook, and occupancy. For every nonempty intersection of an open orthogonal ray from p with D , let $V_{p,d}$ mean that this intersection is occupied. The constraints are

$$\begin{aligned} \neg(K_p \wedge R_p), \quad O_p &\leftrightarrow (K_p \vee R_p), \\ V_{p,d} &\leftrightarrow \bigvee_{q \in D \cap \text{ray}(p,d)} O_q, \\ R_p &\implies \sum_d V_{p,d} \leq 2, \\ R_p \wedge R_q &\implies \bigvee_{s \text{ strictly between } p,q} K_s \quad \text{for aligned } p \neq q, \\ &K_{(0,0)}, \\ K_p &\implies \sum_{q \in D: q-p \text{ is a knight move}} R_q \geq 4 \quad (p \in T). \end{aligned}$$

A ray with empty intersection contributes the constant false. There are no degree constraints on other knights, no knight–knight restriction, and no lower bound of two on the *local* rook ray count. These omissions relax the target, rather than exclude possible counterexamples.

The generator expands the equivalences into implications, uses one guarded clause for each triple of ray flags for the at-most-two constraints, and encodes at least four among t target variables by requiring every $(t - 3)$ -element subset to contain a true variable. Here $t = 4$ at the origin and $t = 6$ at the other distinguished positions.

Clause group	Count
Disjoint types and occupancy equivalences	112
Occupied-ray equivalences	342
Rook occupied-ray upper bounds	54
Intervening-knight conditions	126
Conditional knight lower bounds	64
Origin is a knight	1
Total	699

There are 174 Boolean variables: 84 type/occupancy variables and 90 ray variables. The full DIMACS formula, variable map, and clause explanations are included in the bundle.

3.3 Certificate and verification

The separate DPLL prover produced an UNSAT tree with three nodes: a single split on $K_{(-1,1)}$ and two contradictory leaves. Across the tree there are 151 unit inferences. Every unit inference cites an original clause; every leaf cites an original clause falsified by its branch assignment.

The independent checker `src/verify_unsat.py` verifies the formula’s SHA-256 digest, each unit inference, both branches of the split, and both conflicts. It imports neither the formula generator nor the prover, attack checkers, or Z3. Its inspected output was

```
VERIFIED UNSAT {"conflicts": 2, "nodes": 3,
                "splits": 1, "unit_steps": 151}
```

Independence here means separate algorithms and a proof checker that does not trust the search code. It is not a claim of external peer review or proof-assistant formalization. The human-readable reduction explains why the local UNSAT certificate implies universal finite nonexistence.

4 Calibration, falsification tests, and reproducibility

Two independent attack reconstruction programs were executed. The first uses pairwise knight-coordinate differences and selects the nearest occupied point in each rook ray. The second generates the eight knight jumps explicitly and obtains rook visibility from neighboring entries in sorted occupied rows and columns.

The source’s $N = 1, R = 1$ and $N = 4, R = 1$ illustrations were fetched, visually transcribed, and accepted by both programs [2, 3]. Their original Git blob hashes were reproduced from the decoded

image bytes. The latter illustration has two knights and eight rooks; it is not a solution to the $N = 4, R = 2$ target. All its per-piece outgoing attacks are retained in the accompanying JSON report.

The search also reproduced an $N = 2, R = 3$ calibration with eight knights and six rooks. Direct reconstruction yields 16 knight-to-rook attacks and 18 rook-to-knight attacks. This unequal count explicitly guards against incorrectly imposing reciprocity.

Executed validation included 2,000 seeded random placements, eight translated/reflected/rotated versions of a source example, invalid-piece and blocker tests, five deliberately corrupted certificates (all rejected), and three constraint-removal controls (all verified satisfiable). The full suite ran 20 test methods successfully and regenerated a certificate identical to the bundled one.

Earlier searches and their limits. Exploration used Python 3.13.5 and a locally installed Z3 4.13.3.0 shared library. Unrestricted 8×8 and 16×16 target instances reported UNSAT; symmetry-restricted 12×12 and 16×16 instances also reported UNSAT. A 32×32 attempt was interrupted by the execution time limit and has no reported outcome. None of these bounded searches proves the theorem. Their formulas, seeds, outputs, and limitations are preserved separately.

Run the final verification. The final verification needs only Python 3.9 or newer and its standard library. No network, pip package, or external solver is needed:

```
python3 src/run_all.py
```

For the minimal certificate check alone:

```
python3 src/verify_unsat.py \
  certificates/local_relaxation.cnf \
  certificates/local_unsat_tree.json
```

The bundle also contains `PROOF.md`, `FINITE_REDUCTION.md`, source coordinates and attack reports, executable source code, inspected logs, a checksum manifest, and `progress.json` with the exact verification/resume command.

5 Source conventions and historical status

The target is column $N = 4$, row $R = 2$, in the Knights and Rooks table of Friedman’s February 2007 page [1]. Its accompanying text specifies outgoing attacks on the other type only. The column and row interpretations agree with the independently checked illustrations. The target cell still displayed a question mark when rechecked after the proof was found. A later puzzle post concerns the different $N = 4, R = 1$ case, not this target [4].

Initial and final web searches did not locate an earlier exact-target solution or nonexistence proof. This is not a comprehensive proof of historical priority, and an archival question mark alone does not establish current open status. No priority claim, external review, publication, or personal contact is asserted. The mathematical answer follows from the explicit proof and certificate above, independently of that historical question.

References

- [1] Erich Friedman. *Problem of the Month (February 2007)*, Knights and Rooks table and problem statement. <https://erich-friedman.github.io/mathmagic/0207.html>. Accessed and rechecked in this research session, 13 September 2026.
- [2] Erich Friedman. $N = 1, R = 1$ source illustration. <https://erich-friedman.github.io/mathmagic/0207/N1R1.gif>.
- [3] Erich Friedman. $N = 4, R = 1$ source illustration. <https://erich-friedman.github.io/mathmagic/0207/N4R1.gif>.
- [4] Will.Octagon.Gibson. *Where should the chess pieces be placed in 4x4, 6x6 and 9x9 chessboards?* Puzzling Stack Exchange, 26 August 2024; edited 29 May 2025. Puzzle 2 is $N = 4, R = 1$. <https://puzzling.stackexchange.com/questions/128060/>.